Hochschule Karlsruhe
University of Applied Sciences

Fakultät für **Informatik**
und **Wirtschaftsinformatik**

Dipl. WiInf. Lars Thielemann
Prof. Dr. Heiko Körner

🧪 Tag 3 – Inventar-App

Ziel der Übung

Erstelle eine kleine **Inventar-App** (Department, Person, Device, Assignment), bei der

- das **DB-Schema** über ein **Init-SQL** im Projekt definiert wird,
- zentrale **Domänenregeln** (z. B. max. eine aktive Ausleihe) technisch umgesetzt sind,
- du beim Vorführen **Branch-Wechsel** und wichtige Teile der App **erklären** kannst.

Ausgangsmaterial

Für die Übung nutzt ihr als **Arbeitsbasis** den **inventar-starter**, der auf der Vorarbeit von Tag 2 aufsetzt.

Als **Referenzprojekt** steht euch der **grades-starter** zur Verfügung.

Wichtig: Du arbeitest im **inventar-starter**!

Abgabe

Ihr zeigt in der Gruppe:

1. **Lauffähige Inventar-App**
   - Health (`/health`) und API-Doku (`/docs`) funktionieren.
2. **Inventarverwaltung**
   - Inventar-Schema liegt als **Init-SQL** im Projekt (z. B. `db/init/010_inventory_schema.sql`).
   - CRUD für Devices
   - Erstellen und Auflisten von Assigments
   - Rückgabe von Assigments
   - Domänenregeln werden eingehalten
3. **MQTT**
   - MQTT-Events für Ausleihen.

- MQTT-Chat zwischen zwei Terminals (oder zwei Personen) mit `mosquitto_pub` / `mosquitto_sub`.

Voraussetzungen

- `inventar-starter.zip` als lauffähige Arbeitsbasis mit Docker, Postgres, MQTT, /health und /docs, basierend auf dem Repo von Tag 2.
- `grades-starter.zip` als Blaupause für technische Muster , z. B.:
  * Struktur von backend/app
  * SQL-Initialisierung
  * Form-Verarbeitung mit FastAPI
  * Templates mit Jinja
  * Teil-Updates mit htmx
  * einfache Pydantic-Modelle

A) Inventar-Branch für Tag 3 anlegen

`Zeitbudget: ca. 15–20 min`

Ziel: Ihr legt einen sauberen, gemeinsamen Ausgangspunkt für Tag 3 an. n.

1. Wechselt zunächst in eurem Repository auf main.

2. Prüft kurz euren derzeitigen Arbeitsstand:
   - Gibt es noch nicht commitete Änderungen?
   - Falls ja: erst committen, stashen oder bewusst sichern.
3. Übernehmt anschließend den bereitgestellten **inventar-starter** in euer Repository. Dieser Starter ist ab jetzt eure Arbeitsbasis für Tag 3.

4. Startet das Projekt und prüft es auf Lauffähigkeit:
   - `docker compose up -d --build`
   - Prüfe:
     * `http://localhost:8000/health`
     * `http://localhost:8000/docs`
5. Commitet diesen Stand zunächst auf main, z. B. mit der Nachricht `Startpunkt Tag 3: inventar-starter.`

Wichtig: Für Tag 3 ist der **inventar-starter** euer neuer gemeinsamer Ausgangspunkt. Erst nach diesem Basis-Commit auf main arbeitet ihr in den Folgeaufgaben in Feature-Branches weiter.

B) Referenzprojekt grades-starter verstehen

`Zeitbudget: ca. 20–30 Min.`

Ziel: Ihr schaut euch technische Muster an, die ihr später für eure Inventar-App übernehmen könnt.

1. **grades-starter separat öffnen**
   - `grades-starter.zip` entpacken (z. B. in `grades-starter/`).
   - Der-Container bauen und starten
   - Prüfen:
     * `http://localhost:8000/health`
     * `http://localhost:8000/docs`
     * UI (z. B. `/grades`, `/students`).
2. **Schaut euch gezielt folgende Stellen an:**
   - **DB-Init**: `db/init/001_schema.sql`
     * Welche Tabellen gibt es (`student`, `module`, `grade`)?
     * Wo stehen **Constraints** (UNIQUE, CHECK, FK)?
   - **DB-Zugriff**: `backend/app/db.py`
     * Wie wird `get_conn()` verwendet?
   - **API**:
     * Wo werden Routen für Students / Grades definiert?
   - **Models**: `backend/app/models.py`
     * Wie werden Models aufgebaut?
   - **UI**:
     * Templates unter `backend/app/templates/…` (Listen, Formulare).
3. **Klärt und macht euch Notizen** (z. B. `docs/grades-notes.md`)
   - Wie ist das Projekt strukturiert?
   - Wie werden Formulare verarbeitet?
   - Wie weden Templates eingebunden?
   - Wie sehen einfache CRUD-Flows aus?

Wichtig: Ihr entwickelt nicht im grades-starter weiter. Der **grades-starter** dient nur als Referenzprojekt für Struktur, Formulare, Templates und CRUD-Muster.

C) Feature-Branch für die Inventar-App anlegen

`Zeitbudget: ca. 10–15 Min.`

Ziel: Ihr beginnt die eigentliche Umsetzung der Inventar-App auf Basis eures neuen Tag-3-Startpunkts.

1. **Zurück in euer Inventar-App**

- Geht zurück in euer Inventar-Projekt
- Sicherstellen, dass `main` auf dem soeben erzeugten Stand ist

2. **Neuer Branch für die Inventar-App**
   - z.B. als `feat/inventar-app`
   - (Optional) direkt pushen
3. **Arbeiten im Feature-Branch**

   Wie üblich in Branches:
   - fachliche oder technische Änderungen in einem Feature-Branch,
   - sinnvolle Commits,
   - anschließend zurück nach `main`

D) Inventar-Schema als Init-SQL

`Zeitbudget: ca. 30 min`

Ziel: DB-Schema aufbauen.

Wichtig: Keine Tabellen per Hand in der DB anlegen**, sondern über eine** Init-SQL-Datei im Projekt**.**

1. **Schemata für die Inventar-Domäne anlegen**
   - department
   - person
   - device_type
   - location
   - device
   - assignment
2. **Achte auf:**
   - Primärschlüssel
   - Fremdschlüssel
   - sinnvolle NOT NULL-Constraints
   - eindeutige Seriennummern für Devices
3. **Einfache Seed-Daten**
   - 2–3 `device_type'` (z. B. „Laptop“, „Monitor“, „Handscanner“).
   - 2-3 `location` (z.B. „Gebäude E“, „Gebäude F“, „Gebäude H“).
   - Eine Handvoll `person`-, `device` und `assigment`-Einträge.
4. **Compose neu starten & Schema prüfen**
   - `docker compose down`
   - `docker compose up -d --build`
   - Mit psql im DB-Container nachsehen:

```
docker compose exec db psql -U appuser -d appdb -c "\dt"
docker compose exec db psql -U appuser -d appdb -c "\d device"
# oder mit
docker compose exec db psql -U appuser -d appdb
\dt
\d device
select * from device;
\q
```

- Ggf. Fehler im Init-SQL korrigieren und Container neu starten.

5. **Commit**
   - Änderungen adden und committen.

E) API & Domänenregeln umsetzen

`Zeitbudget: ca. 45–60 min`

Ziel: Ihr baut die ersten fachlichen Endpunkte für eure Inventar-App.

1. Ergänzt in der API erste **Endpunkte für Devices**, z. B.:
   - **GET** `/devices`
   - **POST** `/devices`
2. Ergänzt anschließend **Endpunkte für Assignments**, z. B.:
   - **GET** `/assignments/active`
   - **POST** `/assignments`
   - **POST** `/assignments/{id}/return`
3. Setzt dabei mindestens folgende **Domänenregeln** um:
   - **IR-01**: `serial_number` ist eindeutig
   - **IR-02**: Ein Device darf maximal eine aktive Ausleihe haben
   - **IR-03**: `returned_at` darf nicht vor `issued_at` liegen
4. **Reagiert bei Fehlern** mit sinnvollen Statuscodes, z. B.:
   - `409` Conflict bei doppelter Seriennummer
   - `409` Conflict bei zweiter aktiver Ausleihe
   - `422` bei ungültiger Zeitlogik
5. **Testet** die Endpunkte über `/docs`.

Der grades-starter kann euch hier als Referenz helfen:

- Wie werden Formulardaten verarbeitet?
- Wie werden DB-Zugriffe strukturiert?
- Wie sehen einfache CRUD-Flows aus?

6. **Commit**

F) Einfache Inventar-UI

`Zeitbudget: ca. 45 min`

Ziel: Ihr macht eure Inventar-App über eine einfache Oberfläche benutzbar.

1. **Inventar-Übersichtsseite**
   - Route `/inventory` verfügbar machen.
   - Zeigt dort mindestens eine Liste aller Devices mit
     * ihrem Typ
     * ihrem Standort
     * ihrem Status
   - Hinweis, ob das Gerät „frei“ oder „ausgeliehen“ ist.
2. **Formular „Gerät hinzufügen“**
   - Einfaches Formular zum Anlegen neuer Devices:
3. **Formular „Gerät ausgeben / zurückgeben“**
   - Dropdown für `person` und `device` (nur verfügbare Devices).
   - Button „Ausgeben“ → `POST /assignments`.
   - In der Liste der aktiven Ausleihen ein Button „Zurückgeben“ → `POST /assignments/{id}/return`.
4. **Commit**
   - Template- und Routing-Änderungen committen.

G) MQTT-Events für Ausleihen

`Zeitbudget: ca. 20 min`

Ziel: Ihr nutzt MQTT jetzt in eurer eigenen Domäne.

1. **Überlegt** euch **sinnvolle Topics**, z.B.:
   - `inventory/assignments/returned`
   - `inventory/assignments/issued`
2. **Sendet** bei einer erfolgreichen Ausgabe oder Rückgabe ein MQTT-Event.
3. **Testet** das Verhalten **mit** einem **Subscriber**, z.B:

   ```
   mosquitto_sub -h localhost -t 'inventory/assignments/#' -v
   ```

4. **Prüft**, ob eure Events beim Ausführen eurer Flows sichtbar werden.
5. **Commit**

H) Prüfliste & Präsentation

Prüfliste am Ende

- [ ] `inventar-starter` wurde als Arbeitsbasis übernommen
- [ ] Ein sauberer Basisstand wurde zunächst auf `main` committed.
- [ ] die eigentliche Arbeit erfolgt in einem Feature-Branch.
- [ ] Init-SQL für Inventar ist angelegt und erzeugt alle Tabellen.
- [ ] REST-Endpunkte für Devices & Assignments funktionieren inkl. Domänenregeln.
- [ ] Mindestens eine einfache UI-Seite für Inventar ist vorhanden.
- [ ] MQTT wird für mindestens ein Inventar-Ereignis genutzt
- [ ] Ihr könnt Branch-Wechsel, Schema, API und einen UI-Flow erklären.

Ablauf in der Präsentation

1. Zeigt kurz eure laufende App.
2. Zeigt `/health`, `/docs` und eure Inventar-Seite.
3. Führt einen einfachen Flow vor:
   - Device anlegen
   - Device ausgeben
   - Device zurückgeben
4. Erklärt an 2–3 Stellen im Code:
   - eine Stelle im SQL-Schema
   - einen API-Endpunkt
   - eine umgesetzte Domänenregel

LÖSUNGEN ZU A BIS H

Inventar-App: Umsetzung und Erklärung

Die sieben ursprünglichen Aufgabenseiten stehen vor diesem Lösungsteil. Die praktische Umsetzung liegt im beigefügten Projekt inventar-starter. Das bereitgestellte grades-starter.zip wurde als technische Referenz ausgewertet und nicht verändert.

Bereich	Ergebnis
Datenbank	Sechs Tabellen, Seed-Daten und alle drei geforderten Domänenregeln.
API und Oberfläche	Geräte-CRUD, Ausgeben, aktive Ausleihen, Rückgabe und Inventarübersicht.
MQTT	Ereignisse nach Ausgabe/Rückgabe; Befehle für Subscriber und Chat.
Prüfung	10 Offline-Tests bestanden; echte DB-/MQTT-Integration noch offen.

A und C | Ausgangspunkt und Branch

Beim Beginn war der Arbeitsbaum sauber. Der vorhandene Basis-Commit 1e71dba heißt „Initial Commit Tag-3 inventar-starter“ und lag auf Tag-3. Die Umsetzung entstand darauf im Feature-Branch codex/inventar-app.

Der in der Aufgabe verlangte Basis-Commit auf main wurde damit noch nicht hergestellt. Die bestehende Historie wurde nicht umgeschrieben. Das Team kann den Basisstand nach gemeinsamer Prüfung regulär nach main integrieren. Es wurde nichts gepusht oder gemergt.

```
git status
git branch --show-current
git log --oneline --all --graph
# Erst nach dem Sichern der Änderungen wechseln:
git switch Tag-3
git switch codex/inventar-app
```

Ein Branch-Wechsel ändert die Projektdateien. Er setzt keine Datenbank zurück und führt keine SQL-Migration aus. Die Datei db/init/010_inventory_schema.sql eignet sich, um den Unterschied zwischen Starter und Lösung zu zeigen.

B | REFERENZPROJEKT VERSTEHEN

Muster aus dem grades-starter

Datei / Bereich	Beobachtung
backend/app/main.py	FastAPI-App, Students-/Grades-Routen, Formulare, Template-Aufrufe und MQTT-Publish.
backend/app/db.py	get_conn() öffnet psycopg-Verbindungen mit dict_row und autocommit=True.
backend/app/models.py	Pydantic-Modelle wie StudentCreate, StudentUpdate, ModuleCreate, GradeCreate und Grade.
db/init/001_schema.sql	Tabellen student, module und grade; Primärschlüssel, UNIQUE, CHECK und Fremdschlüssel.
templates/	students/index.html und edit.html; grades/index.html und _list.html für Listen, Formulare und Teil-Updates.

Formulare und Templates

POST /students liest Felder mit Form(...), baut ein StudentCreate-Modell und führt ein parametrisiertes INSERT aus. Bearbeiten und Löschen erfolgen in eigenen POST-Routen. Danach folgt eine aktualisierte Ansicht oder ein Redirect mit Status 303.

Jinja2Templates bindet den Template-Ordner ein. TemplateResponse übergibt Request und Daten. Schleifen erzeugen Tabellen und Dropdowns. Die htmx-Route POST /grades/htmx liefert nur das Teil-Template grades/_list.html.

Constraints im Referenzschema

Matrikelnummer und Modulname sind eindeutig. Das Semester ist per CHECK auf 1 bis 12 begrenzt. grade referenziert student und module; ON DELETE CASCADE löscht abhängige Noten mit. In der Inventar-App bleibt Ausleihhistorie dagegen erhalten und verhindert das Löschen des betreffenden Geräts.

Übertragung auf die Lösung

Die Inventar-App verwendet denselben Stack. Datenoperationen liegen gesammelt in services.py. DB-Verbindungen nutzen Transaktionen, damit ein MQTT-Ereignis erst nach dem Commit gesendet wird. Die Oberfläche wandelt FormData in JSON um und ruft dieselben API-Endpunkte wie /docs auf; htmx ist dafür nicht erforderlich.

Ausführliche Notizen: docs/grades-notes.md. Der Referenz-Stack konnte wegen der nicht erreichbaren Docker-Engine hier nicht gestartet werden.

D | INIT-SQL

Datenmodell und Beispieldaten

Tabelle	Wichtige Felder / Beziehungen
department	department_id (PK), eindeutiger Name
person	person_id (PK), department_id (FK), Vor-/Nachname, eindeutige E-Mail
device_type	device_type_id (PK), eindeutiger Name
location	location_id (PK), eindeutiger Name
device	device_id (PK), Seriennummer, Typ/Standort (FK), Status, Notiz
assignment	assignment_id (PK), device_id/person_id (FK), issued_at, returned_at

Die Datei db/init/010_inventory_schema.sql erzeugt das komplette Inventarschema in einer Transaktion. Pflichtfelder sind NOT NULL. issued_at und returned_at sind timestamptz. Die Verfügbarkeit wird aus aktiven Ausleihen abgeleitet; der Gerätestatus beschreibt Einsatzbereitschaft, Wartung oder Ausmusterung.

Drei Regeln direkt in PostgreSQL

```
CONSTRAINT device_serial_number_key UNIQUE (serial_number)

CREATE UNIQUE INDEX assignment_one_active_device
ON assignment(device_id) WHERE returned_at IS NULL;

CHECK (returned_at IS NULL OR returned_at >= issued_at)
```

Der partielle Index berücksichtigt nur offene Ausleihen. Ein Gerät darf beliebig viele abgeschlossene Ausleihen haben, aber höchstens eine offene. Der Schutz gilt auch für gleichzeitige Requests und direkte SQL-Zugriffe.

Seed-Daten und Schema prüfen

Enthalten sind zwei Abteilungen, drei Personen, drei Gerätetypen, drei Standorte, vier Geräte und zwei Ausleihen: eine aktiv und eine beendet. Typen: Laptop, Monitor, Handscanner. Standorte: Gebäude E, F und H.

```
docker compose exec db psql -U appuser -d appdb -c "\dt"
docker compose exec db psql -U appuser -d appdb -c "\d device"
docker compose exec db psql -U appuser -d appdb -c "\d assignment"
```

Wichtig: Init-SQL läuft nur beim Anlegen einer neuen Datenbank. Ein normaler Neustart führt es nicht erneut aus. Das README beschreibt das einmalige Einspielen in eine vorhandene Datenbank, in der die Inventartabellen noch fehlen. Vorhandene Daten müssen dafür nicht gelöscht werden.

Endpunkte und Fehlerverhalten

Methode / Route	Verhalten
GET /devices	Alle Geräte mit Typ, Standort, Status und Verfügbarkeit.
POST /devices	Gerät anlegen: 201; doppelte Seriennummer: 409.
GET /devices/{id}	Einzelnes Gerät lesen; unbekannt: 404.
PUT /devices/{id}	Gerät mit vollständigem Body ändern.
DELETE /devices/{id}	Ungenutztes Gerät löschen: 204; vorhandene Historie: 409.
GET /assignments	Alle Ausleihen einschließlich abgeschlossener Historie.
GET /assignments/active	Nur Ausleihen mit returned_at IS NULL.
POST /assignments	Ausgeben: 201; bereits ausgeliehen/nicht einsatzbereit: 409.
POST /assignments/{id}/return	Zurückgeben: 200; ungültige Zeit: 422; erneut: 409.

Beispielanfragen über /docs

```
POST /devices
{"serial_number":"DEMO-001","device_type_id":1,
 "location_id":1,"status":"ready","note":"Vorführgerät"}

POST /assignments
{"device_id":5,"person_id":1}

POST /assignments/3/return
{}
```

Die IDs 5 und 3 sind Beispiele: Immer die tatsächlich zurückgegebenen IDs verwenden. Ohne Zeitangabe setzt PostgreSQL die aktuelle Zeit. Eigene Zeitangaben benötigen eine Zeitzone, zum Beispiel 2026-10-24T12:00:00+02:00.

Validierung und Transaktionen

Pydantic validiert Pflichtfelder, positive IDs, erlaubte Statuswerte, Zeitzonen und leere Seriennummern. Fremdschlüsselfehler bei ungültigen Referenzen ergeben 422. SQL-Parameter werden separat übergeben. FOR UPDATE koordiniert die Ausgabe mit Statusänderungen und schützt Rückgaben vor paralleler Wiederholung.

Die drei fachlichen Regeln werden zusätzlich auf DB-Ebene erzwungen. Die API übersetzt Constraint-Verletzungen in verständliche Antworten; nur erfolgreiche Transaktionen lösen MQTT-Ereignisse aus.

F | INVENTAR-OBERFLÄCHE

Geräte verwalten und ausleihen

GET /inventory rendert backend/app/templates/inventory.html. Die Seite enthält eine Geräteübersicht, ein Formular zum Anlegen, eine Bearbeitungsmaske sowie Ausgabe und Rückgabe. Alle Funktionen sind auch über /docs erreichbar.

Anzeige / Aktion	Umsetzung
Geräteliste	Seriennummer, Notiz, Typ, Standort, Status und Verfügbarkeit.
Gerät hinzufügen	Typ und Standort aus DB-Dropdowns; Status und optionale Notiz.
Bearbeiten / Löschen	PUT bzw. DELETE auf /devices/{id}; Fehler bleiben sichtbar.
Gerät ausgeben	Person und nur freie, einsatzbereite Geräte auswählen.
Aktive Ausleihen	Person, Seriennummer, Ausgabezeit in UTC und Rückgabebutton.
Rückgabe	POST auf /assignments/{id}/return mit leerem JSON-Objekt.

Formularverarbeitung

JavaScript fängt das Absenden ab, sammelt Felder mit FormData und sendet JSON per fetch an die API. IDs werden zu Zahlen konvertiert. Nach Erfolg lädt die Seite neu; bei Fehlern wird die API-Meldung angezeigt. JavaScript ist für die Formulare erforderlich. Es gibt keine externe CDN-Abhängigkeit.

Jinja maskiert angezeigte Benutzereingaben. Auch die in JavaScript verwendeten Gerätedaten werden mit tojson sicher eingebunden. Eine Oberfläche allein verhindert keine doppelte Ausleihe: Die Datenbank prüft den Vorgang in jedem Fall erneut.

Vorführ-Flow

1. Unter /inventory ein neues Gerät DEMO-001 anlegen.
2. Das Gerät und eine Person im Ausgabeformular auswählen.
3. Nach „Ausgeben“ erscheint es als ausgeliehen und fehlt im Dropdown.
4. Unter den aktiven Ausleihen „Zurückgeben“ wählen.
5. Das Gerät ist wieder frei und erneut ausleihbar.

Status und Historie erklären

„Einsatzbereit“ beschreibt den Gerätezustand; „frei“ oder „ausgeliehen“ beschreibt die aktuelle Verfügbarkeit. Wartungsgeräte werden nicht angeboten. Geräte mit Ausleihhistorie lassen sich nicht löschen; sie können nach der Rückgabe ausgemustert werden. So bleiben frühere Ausleihen nachvollziehbar.

G | MQTT

Ereignisse und Zwei-Terminal-Chat

Nach dem Commit einer erfolgreichen Ausgabe bzw. Rückgabe sendet mqtt_integration.py ein JSON-Ereignis mit QoS 1 und ohne retain. Abgelehnte API-Aufrufe erzeugen kein Erfolgsereignis.

Topic	Auslöser
inventory/assignments/issued	Gerät erfolgreich an Person ausgegeben.
inventory/assignments/returned	Aktive Ausleihe erfolgreich zurückgegeben.

```
{
  "assignment_id": 3,
  "device_id": 5,
  "person_id": 1,
  "issued_at": "2026-10-24T10:00:00+00:00",
  "returned_at": null
}
```

Bei der Rückgabe enthält returned_at den tatsächlichen Rückgabezeitpunkt. Die IDs identifizieren den Vorgang. Ein Ereignis enthält keine Namen oder E-Mail-Adressen.

Ausleihereignisse empfangen

```
docker compose exec mqtt mosquitto_sub -h localhost `
  -t "inventory/assignments/#" -v
```

Den Befehl in PowerShell in einer Zeile eingeben. Anschließend in der UI ausgeben und zurückgeben. Im Subscriber sollen die beiden Topics mit JSON-Nutzdaten erscheinen.

Chat zwischen zwei Terminals

```
# Terminal 1: empfangen
docker compose exec mqtt mosquitto_sub -h localhost `
  -t "inventory/chat/tag3" -v

# Terminal 2: senden
docker compose exec mqtt mosquitto_pub -h localhost `
  -t "inventory/chat/tag3" -m "Selim: Hallo aus Terminal 2"
```

Auch diese Befehle jeweils in einer Zeile eingeben. localhost bezeichnet hier den Broker im Container. Mit lokal installierten mosquitto-Tools können die Befehle ohne den Docker-Prefix über Host-Port 1883 ausgeführt werden.

Ausfallverhalten: Ein Broker-Ausfall wird geloggt; der bereits gespeicherte DB-Vorgang bleibt gültig. Es gibt keine dauerhafte Warteschlange und keinen automatischen späteren Versand. QoS 1 kann Duplikate liefern; es ist keine Exactly-once-Garantie.

H | PRÜFUNG UND PRÄSENTATION

Nachweise und noch offene Live-Prüfung

Tatsächlich geprüft: 10 Offline-Tests bestanden, 1 Integrationstest übersprungen. Die Tests prüfen API-Schema, Eingabevalidierung, Zeitzonen, MQTT-Ausfallbehandlung, HTML-Maskierung und das Geräte-Dropdown mit simulierten Daten. Alle Python-Dateien sind syntaktisch geprüft. Beide Compose-Konfigurationen und git diff --check sind gültig.

Noch nicht live bestätigt: Start der App und des Referenzprojekts, Schema-Initialisierung, die drei Regeln gegen echtes PostgreSQL, parallele Ausgabe, MQTT-Zustellung und Chat. Die Docker-Engine war in dieser Umgebung auch nach Startversuch nicht erreichbar. Das Ergebnis wird daher nicht als vollständig laufzeitgeprüft ausgegeben.

Start und vollständigen Integrationstest ausführen

Alle Befehle aus inventar-starter/inventar-starter ausführen:

```
docker compose up -d --build
# Browser: /health, /docs und /inventory auf Port 8000

# Separater Test-Stack ohne belegte Host-Ports:
docker compose -p inventory-tests -f compose.test.yml up `
  --build --abort-on-container-exit --exit-code-from test

# Nur den isolierten Test-Stack danach aufräumen:
docker compose -p inventory-tests -f compose.test.yml down -v
```

Die langen Befehle in PowerShell jeweils in einer Zeile eingeben. Der Test-Stack prüft echtes CRUD, IR-01/02/03, parallele Ausgabe und die beiden MQTT-Ereignisse. Er verwendet eigene Testdaten. Die normale App behält ihre Daten in einem benannten Volume.

Präsentation in sechs Schritten

1. /health, /docs und /inventory zeigen.
2. Neues Gerät anlegen, ausgeben und zurückgeben.
3. Im Subscriber die beiden Ereignisse zeigen.
4. Eine doppelte Seriennummer bzw. zweite aktive Ausleihe mit 409 vorführen.
5. SQL-Index, API-Route und gemeinsame Fachfunktion erläutern.
6. Chat und Git-Branch-Unterschiede zeigen; den noch offenen main-Basisstand ansprechen.

Orientierung im Projekt

README.md: Start, API, MQTT und Testbefehle.
docs/grades-notes.md: Analyse der Referenz.
docs/aufgaben-loesung.md: Bearbeitung A bis H.
docs/test-ergebnis.txt: Tatsächlicher Prüfstand.
tests/: Offline- und Integrationstests.